

Computer Peripherals - Arduino Lab 2

1. Pulse Width Modulation (PWM)

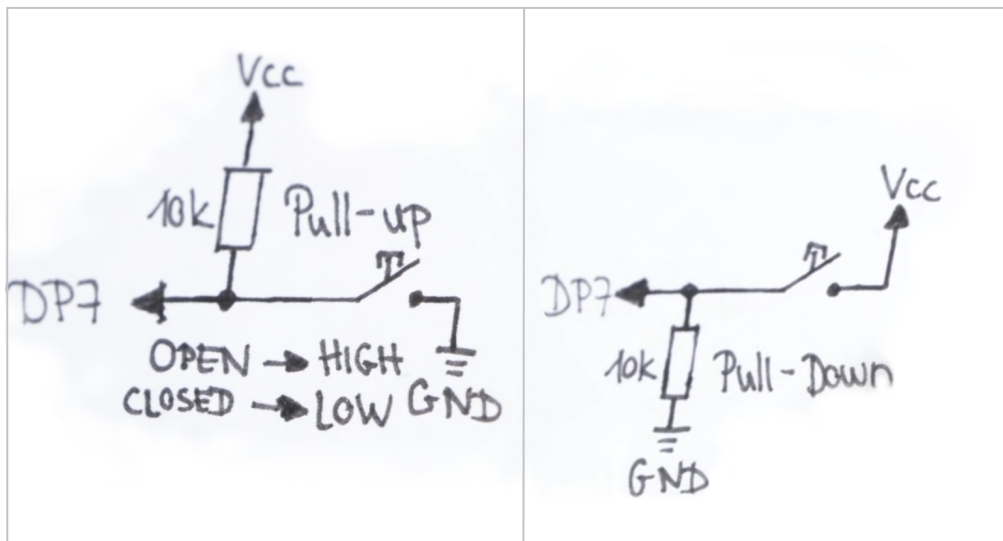
The Arduino board has the following main functions: digital input, digital output, pwm output, and analog input. So far, we covered in the previous lab only one of them, the digital output. Before we are going to the task let's take a look, at each of them.

For debugging purposes we can initialize the *Serial Monitor* with a baud rate in the setup section (like *Serial.begin(9600)*), this will be useful when we want to check a variable while the code runs. To see our variable on the Serial Monitor we need to use the *Serial.println()* function in our loop section.

DIGITAL INPUT

We can set a digital pin as input by using the *pinMode()* function like in the previous lab, but now should be set to *INPUT*. To read the value of an input we can use the *digitalRead()* function.

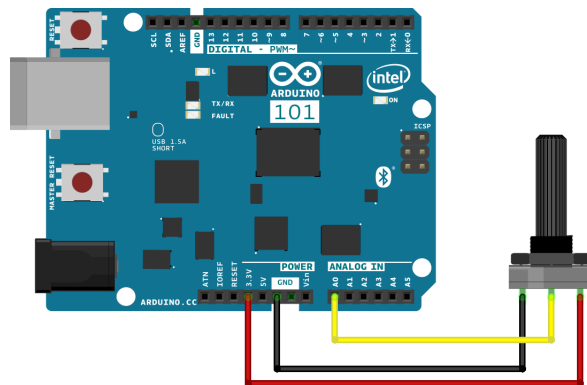
If the wire at the input of a digital pin is floating, then the returned value might be invalid, to avoid these wrong values, we can use a 10k resistance in a *pull-up* or *pull-down* configuration like in the figure, or simply we can use the integrated pull-up resistor of the Arduino board by declaring our input as an *INPUT_PULLUP*.



ANALOG INPUT

While the digital input can have only two states (high/low, 0/1) at the analog input we can read 1024 discrete values. By connecting a 5V analog sensor output to one of our analog inputs on the Arduino board, by using the (*AnalogRead()*) function we can read the voltage applied to the analog input pins. The returned values are scaled (because of the 10 bit analog to digital converter), so this function returns a number between 0 and 1023, which represents voltages between 0 and 5 volts. For example, if there is a voltage of 2.5 V applied to pin 0, *analogRead(0)* returns 512.

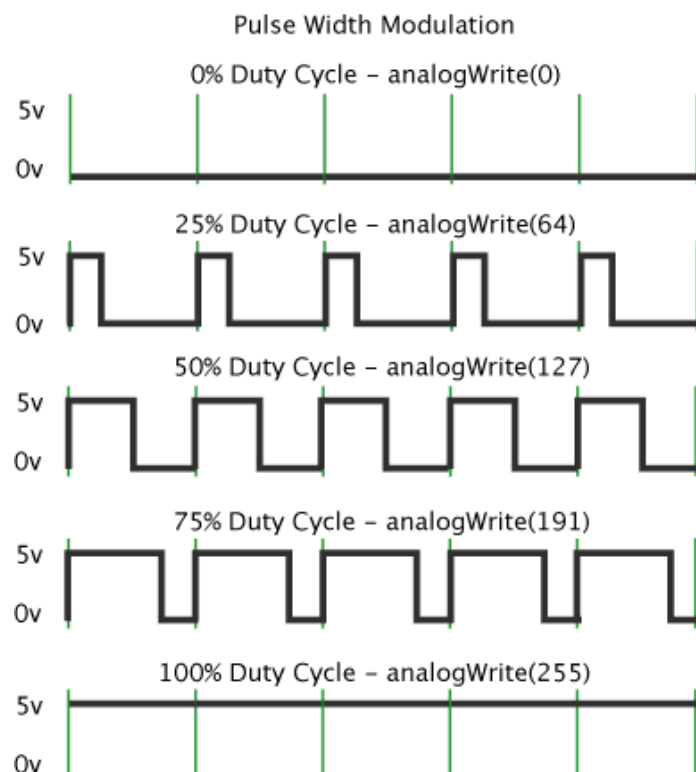
An example connection used to read the value of a rotary potentiometer, which acts as a voltage divider can be seen in the figure below.



PWM OUTPUT

By definition the Pulse Width Modulation, or PWM, is a technique for getting analog results with digital means. Digital control is used to create a square wave, a signal switched between on and off (in a reasonable frequency, in our case usually 490 Hz).

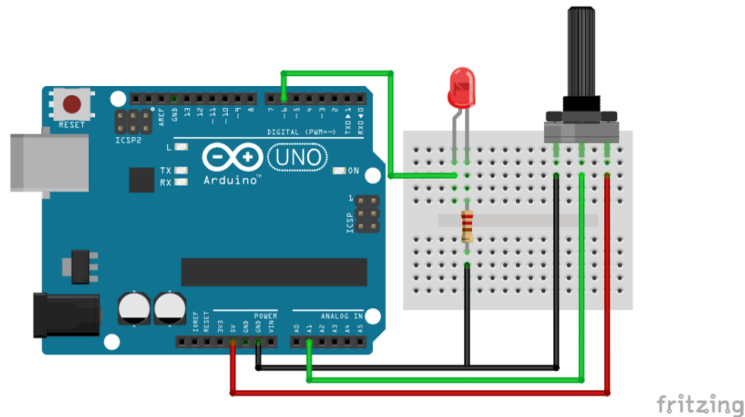
The duration of "on time" is called the pulse width. To get varying analog values, you change, or modulate, that pulse width. If you repeat this on-off pattern fast enough with an LED for example, the result is as if the signal is a steady voltage between 0 and Vcc controlling the brightness of the LED. In the graphic below, the green lines



represent a regular time period. This duration or period is the inverse of the PWM frequency. In other words, with Arduino's PWM frequency at about 500Hz, the green lines would measure 2 milliseconds each. A call to `analogWrite()` is on a scale of 0 - 255, such that `analogWrite(255)` requests a 100% duty cycle (always on), and `analogWrite(127)` is a 50% duty cycle (on half the time) for example.

2. The Task

Please realize the following Arduino setup is based on the provided schematic. Then write a program that controls



the intensity of the LED, based on the potentiometer position. A pseudo-code is provided in the algorithm below.

Algorithm 1 AnalogInOutSerial

define:

//Analog input pin

//Analog output pin

// variable for read from the pot

// variable for output to the PWM

setup:

// initialize serial communication

loop:

// read the analog in value:

// map it to the range of the analog out:

// change the analog out value

// print the results to the Serial Monitor

// wait 2 milliseconds before the next loop

Bibliography

This lab guide is based on *GreatScott* Youtube channel content, and *create.arduino.cc* website contents.